

08 — Administrator Guide

This guide is for the **Admin** permission group — the platform administrator who can view and manage **every** event. It is intentionally separate from the [user guide](#).

Who is an admin?

An admin is a Cognito user that belongs to the **admin** group in the **SeatMe-Users** user pool. The deployment seeds one admin automatically (see [../10-live-link-and-credentials](#) for the credentials).

How admin access is enforced

- The HTTP API is intentionally open so the public RSVP page and the no-login preview link keep working.
- The admin endpoint `GET /admin/hosts` is therefore enforced **inside its Lambda** (`list_hosts`): it reads the caller's Cognito **access token** from the `Authorization: Bearer <token>` header, validates it with `GetUser`, and confirms the user is in the **admin** group with `AdminListGroupsForUser`. Non-admins get **403**; missing/invalid tokens get **401**.
- **Write endpoints** (create / update / delete a host or guest, set tables, generate seating, send invitations) are protected the same way via the shared `_common.require_owner` helper: the caller must own the targeted event or be an admin. An admin's token therefore lets them edit **any** host's event, while an anonymous preview is read-only. Read endpoints stay public so RSVP and preview links work.

Signing in as admin

1. Go to the **Log in** page and sign in with the admin email and password.
2. You are routed automatically to `admin.html` (the admin portal). The app detects the **admin** group from the Id token and redirects you there.

Using the admin portal (`admin.html`, Feature F18)

- The portal lists **all events** on the platform: event name, host email, date, location, and guest count.
- **Open** — opens any event in the standard host dashboard (`host.html?host=<event-email>`) so you can review or edit it.
- **Delete** — removes an event and all its guests (same effect as a host deleting their own event). Use with care; this cannot be undone.

Managing admins and users

Admin group membership is managed in Amazon Cognito (no in-app screen):

```
# Find the pool id
aws cognito-idp list-user-pools --max-results 50 \
  --query "UserPools[?Name=='SeatMe-Users'].Id" --output text

# Promote an existing user to admin
```

```
aws cognito-idp admin-add-user-to-group \
  --user-pool-id <POOL_ID> --username <email> --group-name admin

# Remove admin rights
aws cognito-idp admin-remove-user-from-group \
  --user-pool-id <POOL_ID> --username <email> --group-name admin
```

The deploy script `backend/deploy/setup_cognito.py` creates the `admin` and `host` groups and the seeded admin/host users automatically; re-running it is safe (it is idempotent). Self-service sign-ups are added to the `host` group automatically by the `auth_post_confirmation` Cognito Post-Confirmation trigger, so every account belongs to an explicit permission group.

Operational tasks

Task	How
Redeploy everything	<code>python redeploy_all.py</code> from the repo root
Reset a host's password	Cognito console, or <code>admin-set-user-password</code>
Inspect data	DynamoDB console, table <code>SeatMe</code>
Rotate the seeded admin password	Set env var <code>SEATME_ADMIN_PASSWORD</code> before deploy, or use <code>admin-set-user-password</code>

Security responsibilities

- Change the seeded admin password before any real use.
- Keep the `admin` group small; every member can read, edit, and delete all events.
- The roadmap item to add a Cognito JWT authorizer to the whole HTTP API would let the token checks move from the Lambdas to the gateway — track this before production use.